

Technical Design Document - Space Shooter

Course: BMI4242 Game Programming **Stack:** Python 3.11+ / Pygame 2.x

1. Introduction

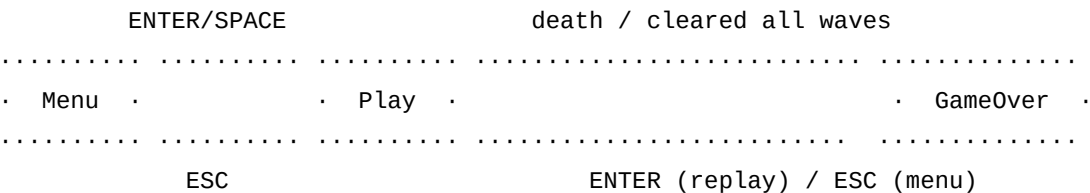
Space Shooter is a 2D arcade shoot-'em-up in the Space Invaders / Galaga tradition. The player pilots a ship at the bottom of the screen and must survive five increasingly aggressive waves of enemies that march in formation, dive along curved paths, and fire downward. Victory is surviving all `TOTAL_WAVES`; defeat is losing all health.

Role	Responsibility (suggested split for a 4-person team)
Member A	Gameplay <code>player.py</code> , input, shooting, <code>play_state.py</code>
Member B	Enemies/AI <code>enemy.py</code> , <code>enemy_ai.py</code> , <code>wave_manager.py</code>
Member C	Engine/Systems <code>object_pool.py</code> , <code>collision.py</code> , <code>animated_sprite.py</code>
Member D	UI/Flow <code>game.py</code> state machine, <code>menu/gameover</code> states, <code>hud.py</code>

2. Software Architecture

2.1 State Machine

`GameManager` owns exactly one active `State` and forwards `handle_event`, `update(dt)` and `draw(surface)` to it. Transitions go through `change_state(name)`, which also fires the new state's `on_enter()` hook.



All three states subclass `State` (`src/states/base_state.py`), so the loop in `GameManager.run()` never special-cases which screen is showing.

2.2 OOP Class Hierarchy

```
pygame.sprite.Sprite
... Player          (movement, shoot cooldown, health, i-frames)
... Bullet          (poolable projectile, up or down)
... Enemy           (formation slot + dive path execution)
... AnimatedSprite  (sprite-sheet frame stepping)
    ... Explosion   (one-shot, poolable)
```

Entities are grouped in `pygame.sprite.Group`'s for batch `update()`/`draw()`.

Raw coordinate math is confined to entity methods and the AI system · never the game loop.

2.3 Orchestration

``GameManager.__init__`` loads every image/sound **once** and stores them in dictionaries. ``PlayState.on_enter()`` builds the world (player, groups, pools, ``FormationManager``, ``WaveManager``, ``HUD``). Each frame ``PlayState.update()``: gather input · step waves · step AI · move projectiles · resolve collisions · check win/lose.

3. Algorithms & Data Structures

3.1 Object Pooling (``systems/object_pool.py``)

A generic ``ObjectPool[T]`` pre-allocates a fixed list of reusable objects whose only requirement is a boolean ``active`` flag. ``acquire()`` linear-scans for the first inactive object, marks it active, and returns it (or ``None`` when full); objects return themselves by setting ``active = False`` and calling ``kill()`` to leave their sprite groups. Bullets and explosions are **never** constructed in the loop · eliminating per-frame allocation.

3.2 AABB + Mask Collision (``systems/collision.py``)

Broad phase = Axis-Aligned Bounding Box overlap. Two boxes ``A`` and ``B`` collide iff ``A.left < B.right · A.right > B.left · A.top < B.bottom · A.bottom > B.top`` (pygame's ``Rect.colliderect``). Narrow phase refines true hits with ``pygame.mask.collide_mask``, comparing per-pixel masks so transparent corners of sprites don't register false hits. We wrap ``groupcollide``/``spritecollide`` with ``dokill=False`` so the pool · not pygame · controls object lifetimes.

3.3 Enemy AI · Three Phases (``systems/enemy_ai.py``)

1. **Formation:** the whole grid shares one horizontal ``offset_x``. Each frame the block advances by ``FORMATION_SPEED``; when it reaches a wall it reverses and adds ``FORMATION_STEP_DOWN`` to ``offset_y``. Slot centres are computed from ``(row, col)`` + offsets, then each formation enemy is snapped to its slot.
2. **Dive:** with small per-frame probability (capped at ``MAX_CONCURRENT_DIVERS``) an enemy breaks formation. Its path is sinusoidal:
``x = origin_x + A·sin(·)``, ``y += DIVE_SPEED``, with ``· += DIVE_FREQUENCY`` each frame · the classic swooping arc. Past the bottom edge it recycles to its slot from the top.
3. **Shooting:** each enemy fires downward with probability ``ENEMY_SHOOT_CHANCE``. The AI only *reports* muzzle positions; ``PlayState`` pulls the bullet from the pool and plays the SFX, keeping pooling/audio out of the AI policy.

3.4 Sprite-Sheet Animation (`entities/animated\_sprite.py`)

`\_slice\_sheet` cuts a horizontal strip into `width // frame\_width` equal sub-surfaces. Playback is **dt-driven**: `\_elapsed += dt`, and while `\_elapsed < 1000/fps` we advance one frame (looping, or deactivating a one-shot explosion at the end). Using accumulated `dt` instead of wall-clock makes the animation frame-rate independent and deterministically testable.

4. Optimization

- **Object pooling** · less GC pressure. Bullets/explosions are reused, so the allocator and garbage collector stay idle during intense fire · no frame-time spikes from collection pauses at 60 FPS.
- **Sprite-group batch rendering**. `Group.draw()` blits every member in one C-level pass instead of Python-level per-sprite calls, and `Group.update(dt)` batches logic updates.
- **Load-once assets**. All images/sounds are decoded a single time at start-up and cached (`src/assets.py`); the update loop never touches disk. Images are `convert\_alpha()`-ed once so per-blit format conversion is avoided.
- **Broad-then-narrow collision**. Cheap AABB rejects most pairs before the more expensive pixel-mask test runs, keeping collision near O(candidates).

5. Git Strategy

- **Branch model**: `main` (stable, runnable builds only) · `dev` (integration)
- `feature/\*` (e.g. `feature/player-movement`, `feature/enemy-ai`, `feature/object-pooling`, `feature/ui-hud`).
- **Commit conventions**: `feat: add bullet pool`, `fix: collision rect offset`, `docs: update TDD`. Every member lands 10+ commits before the deadline.
- **Merging**: never commit directly to `main`; merge `dev` · `main` only when the build runs without errors.
- **Conflict resolution**: for a conflict in a `.py` file, the original author of that code resolves it.